

LABORATÓRIO DE SISTEMAS DIGITAIS

Matilde A. Costa 2019/2020

- Técnico - Prática : Guilherme Campos
- Prática : Armando Nolasco Pinto (anp@ua.pt)
- Avaliação : $\odot$ Prática = 60%

• Regente : Ioulia Sklionova

Nota mínima : 7,5

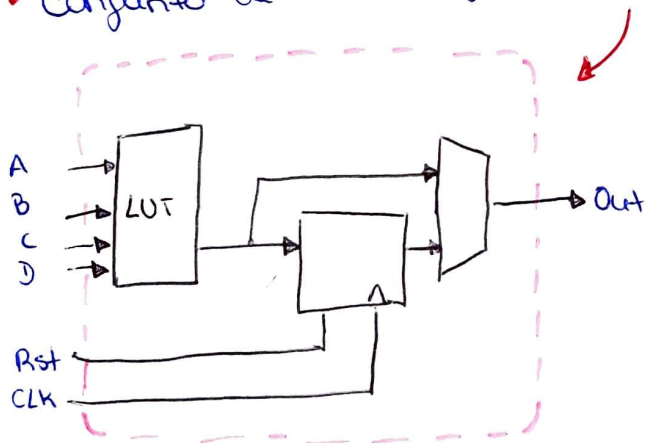
- Pelo menos 2 minitestes $\rightarrow$ 30%
- Mini projeto - 10%
- Projeto final - 50%
- Avaliação nas aulas práticas - 10%
- $\odot$ Teórica = 40%
- 2 testes escritos individuais (20.03 e 22.05)

Breve introdução às FPGA :

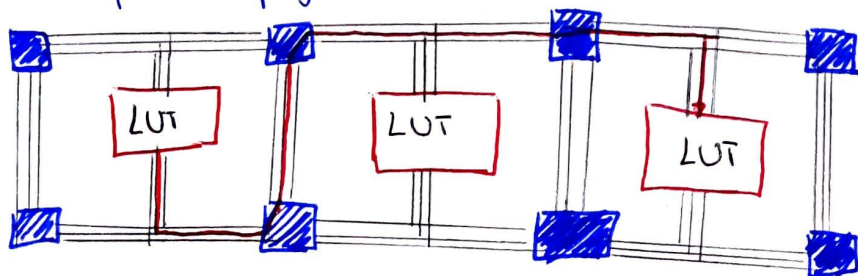
↳ Arquitetura básica:

- FPGA - Field Programmable Gate Array;
- Conjunto de blocos lógicos configuráveis:

LUT $\rightarrow$ LookUp Table

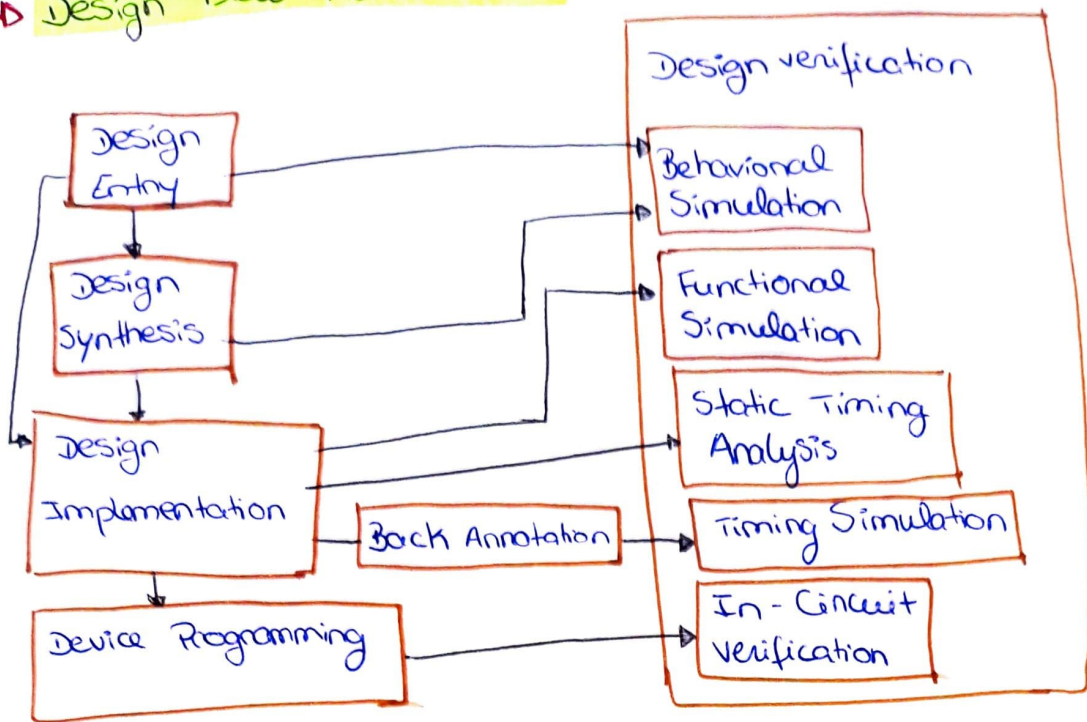


- O posicionamento e interligação de LUT's nos FPGA's é realizado pelas ferramentas de compilação / projeto.



↳ mapeamento através de switch boxes

↳ Design Flow numo FPGA:



- Design entry :

- Linguagens de descrição de hardware;
- Diagramas esquemáticos;
- Diagramas de estado.

- Síntese Lógica:

- Resulta numa netlist (isto é: nos componentes de hardware e nas suas interconexões), que implementa o comportamento e a estrutura modeladas.
- Resultados:
 - Netlist;
 - Estimativas de desempenho do circuito e recursos lógicos necessários

- Implementação (Fit / Place and Route):

- Mapeia a netlist em primitivas da FPGA;
- Posiciona as primitivas da FPGA em localizações específicas;
- Realiza (Encaminha) as interconexões entre as primitivas;
- Resultados:
 - Ficheiro de configurações da FPGA;
 - Relatório sobre os recursos utilizados da FPGA, tempos de atraso e outras métricas (consumo energético,...).

- Programação do dispositivo:

- Transfere o ficheiro de configuração para a FPGA:
 - Realizada através de um software e de um cabo de programação adequado
 - Configuração volátil - SRAM
 - Configuração não volátil - memórias FLASH

Introdução à linguagem VHDL

↳ Aspectos básicos:

- Very High Speed integrated circuits Hardware Description Language
- Não é uma linguagem de programação de software!
- Serve para realizar modelação, simulação e síntese de sistemas digitais
- Permite descrever o comportamento e a estrutura de hardware digital, utilizando construções típicas das linguagens de programação
- Abstrações para as construções típicas do hardware:
 - Entities: interface de um módulo;
 - Architectures: implementação de um módulo;
 - Signals: sinais internos de um módulo;
 - Ports: sinais da interface de um módulo; (entradas/saídas)
 - Tipos de dados orientados para o hardware: para representar níveis lógicos, vetores de bits de diversos tamanhos, noção de tempo, etc ...
 - Concorrência: para modelar o paralelismo do hardware;
 - ...

↳ Tipos de dados em VHDL:

Biblioteca: IEEE STD-LOGIC-1164

• Mais utilizados:

- | | |
|-------------------------------------|--------------------------------|
| → '0' - Nível baixo | → '1' - Nível alto |
| → 'L' - Nível baixo passivo | → 'H' - Nível alto passivo |
| → 'Z' - Tri-State (alta impedância) | → '-' - Don't care |
| → 'X' - Conflito (entre 0 e 1) | → 'u' - Conflito (entre L e H) |
| → 'U' - Não inicializado | |

• Outros:

- unsigned, signed
- Integer
- Enumerated
- Boolean
- character
- time

• Std_logic → 1 bit

• Std_logic_vector → vários bits - barramentos

↳ VHDL - Identificadores

- VHDL é uma linguagem not case - sensitive.
- OS identificadores servem para nomear itens.
- Só podem incluir letras, dígitos e underscore (_)
- Devem começar com uma letra
- Não podem conter o underscore no fim
- Não podem conter dois underscores seguidos
- Ter atenção às palavras reservadas, tanto do VHDL como do Verilog!

Modelação de Componentes Combinatórios

↳ Componentes combinatórios:

• Um componente combinatório é um circuito cujas saídas só dependem do valor das entradas em cada momento.

• Exemplos:

- Portas lógicas elementares
- Multiplexers
- Descodificadores / codificadores
- Comparadores
- Somadores / Subtratores

↳ Arquitetura de um módulo - diversas abordagens:

• **Structural (Estrutural):**

→ Instanciar e interligar as portas lógicas necessárias. Pode-se criar certas portas e instanciá-las as vezes necessárias

• **Behavioral Equations (Equações lógicas):**

→ Escrever a equação lógica para cada saída ligar entradas às saídas

• **Behavioral Assignment (Atribuição Condicional):**

→ Atribuir cada combinação de saída à respetiva combinação de entrada

Nota: No Quartus Prime, este tipo de atribuições condicionais só pode ser usado diretamente no corpo da arquitetura (fora de processos)

- Behavioral Process 1 (Processo com if... then... else):
 - Atribuir cada combinação de entrada ao seu efeito (comportamento), usando a lógica if... then... elsif... then... else.
- Behavioral Process 2 (Processo com case... when):
 - Atribuir cada combinação de entrada ao seu efeito (comportamento), usando a lógica case... when.

↳ Aspectos fundamentais:

- O corpo da arquitetura pode conter:
 - Instanciação e interligação de componentes;
 - Atribuições concorrentes (funções lógicas);
 - Atribuições concorrentes condicionais;
 - Processos;
- Num processo relativo a um componente combinatório:
 - A lista de sensibilidade deve conter todos os sinais aos quais o processo é sensível (entradas que influenciam as saídas).
 - As saídas devem estar especificadas para todas as combinações das entradas (no caso dos condicionais).

Modelação de circuitos aritméticos:

USE IEEE.NUMERIC_STD.ALL;

↳ Semânticas:

- Algumas abordagens de modelação ao nível da arquitetura em VHDL:
 - ① - Instanciar um "full adder" e interligar as portas lógicas necessárias (Estrutural);
 - ② - Escrever as equações lógicas de cada saída;
 - ③ - Escrever a expressão aritmética da saída ($a+b$);

Exemplo ponto ③:

architecture Behavioral of Adder 4 is

signal S_operand0, S_operand1, S_Result: unsigned (4 downto 0);

5 bits

begin

S_operand0 <= '0' & unsigned(operand0);

S_operand1 <= '0' & unsigned(operand1);

S_Result <= S_operand0 + S_operand1;

Result <= std_logic_vector(S_Result(3 downto 0));

CarryOut <= std_logic(S_Result(4));

8-concatenação

Reparar a binário

• Ao usar os tipos `signed` e `unsigned`, o processador interpreta os valores dos inputs de forma diferente, sendo possível realizar uma grande variedade de operações:

- Adição (+);
- Subtração (-);
- Multiplicação (*);
- Divisão (/);
- Resto da divisão inteira (rem).
- (...)

Nota: Adições e subtrações com ou sem sinal representadas em complemento para 2 são realizadas da mesma forma. O mesmo não acontece com as multiplicações e divisões.

↳ Comparadores:

• Da mesma forma que os somadores, podemos usar operadores relacionais com quantidades `signed` e `unsigned`:

```
equal <= '1' when (operand0 = operand1) else  
    '0';  
not equal <= '1' when (operand0 /= operand1) else  
    '0';
```

↳ não precisam
de comparar com
quantidades

```
first greater <= '1' when (signed(operand0) < signed(operand1)) else  
    '0';  
  
(...)
```

Componentes parametrizáveis:

O que são?

• Componentes que possuem algumas características passíveis de serem especificadas (especializadas) quando da sua instânciação (ex: número de bits em somador, multiplexer, etc;

Para que servem?

- Evitar bibliotecas com todas as especializações e variantes possíveis de todos os componentes standard (ex: Somadores de 1, 2, 3, ..., 30, 31, 32 bits)

Como fazer?

- Criação do componente - descrever o comportamento e estrutura genérica do componente baseada em parâmetros genéricos definidos na entidade.
- Utilização do componente - (em VHDL) instanciar o componente, ligando os portos e atribuindo valores concretos às constantes genéricas. (Em diagramas lógicos) Definir um símbolo, instanciar o componente e definir valores concretos para os parâmetros.

Ex: Somador de N bits com carry-out:

entity AdderN is

generic (N : positive := 4)

port (operando: std_logic_vector (N-1 downto 0);

(...)
end AdderN;

Architecture Behavioral of AdderN is

(...)

begin

S_Result <= S_operando + S_operando1;

Result <= std_logic_vector (S_Result (N-1 downto 0));

(...) carry_out <= std_logic (S_Result(N));

↑ inteiro positivo

↑ Valor por emissão do generic

↑ CRIAÇÃO

(...)

adder1: entity work.AdderN (Behavioral)

generic map (N => 4)

port map (operando => A,

operando1 => B,

(...)

↑ INSTANCIÇÃO

Modelação de circuitos sequenciais:

↳ Latch D

```
if (enable = '1') then  
  dataOut <= dataIn;  
end if;
```

→ O facto de não especificar a saída quando enable = '0', faz com que a saída se mantenha a anterior até enable voltar a ser 1.

• Se fizermos a simulação temporal, o sinal dataOut surge inicialmente como indelével ("xxxxx"). Isto acontece quando o enable é inicialmente '0', pois a saída "anterior" não está especificada.

↳ Flip-Flop tipo D:

```
begin  
  process (clk)  
    begin  
      if (clk'event and clk = '1') then  
        dataOut <= dataIn;  
      end if;  
    end process;
```

clk'event and clk = '1' é
equivalente a rising-edge(clk)

clk'event and clk = '0' é
equivalente a falling-edge(clk)

E se quisermos usar um sinal de enable?

```
begin  
  process (clk)  
    begin  
      if (rising-edge(clk)) then  
        if (enable = '1') then  
          dataOut <= dataIn;  
        end if;  
      end if;  
    end process;
```

→ Neste caso, o enable é
Síncrono, porque o clk tem mais prioridade

Reset Assíncrono:

```
process (reset, clk)
begin
  if (reset = '1') then
    dataOut <= '0';
  elsif (rising_edge(clk)) then
    if (enable = '1') then
      dataOut <= dataIn;
    end if;
  end if;
end process;
```

↳ Verificação:

- 1^ª - Reset
- 2^ª - Rising-edge
- 3^ª - Enable

Reset Síncrono:

```
process (clk)
begin
  if (rising_edge(clk)) then
    if (reset = '1') then
      dataOut <= '0';
    elsif (enable = '1') then
      dataOut <= dataIn;
    end if;
  end if;
end process;
```

↳ Verificação:

- 1^ª - rising-edge
- 2^ª - Reset
- 3^ª - Enable

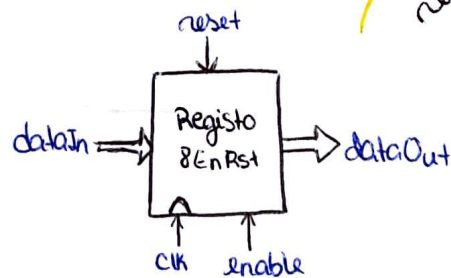
↳ Registro de 8 bits

Exemplo com Reset síncrono:

```
process (clk)
begin
  if (rising_edge(clk)) then
    if (reset = '1') then
      dataOut <= (others => '0');
    elsif (enable = '1') then
      dataOut <= dataIn;
    end if;
  end if;
end process;
```

Verificação:

- 1^ª - rising-edge
- 2^ª - reset
- 3^ª - Enable



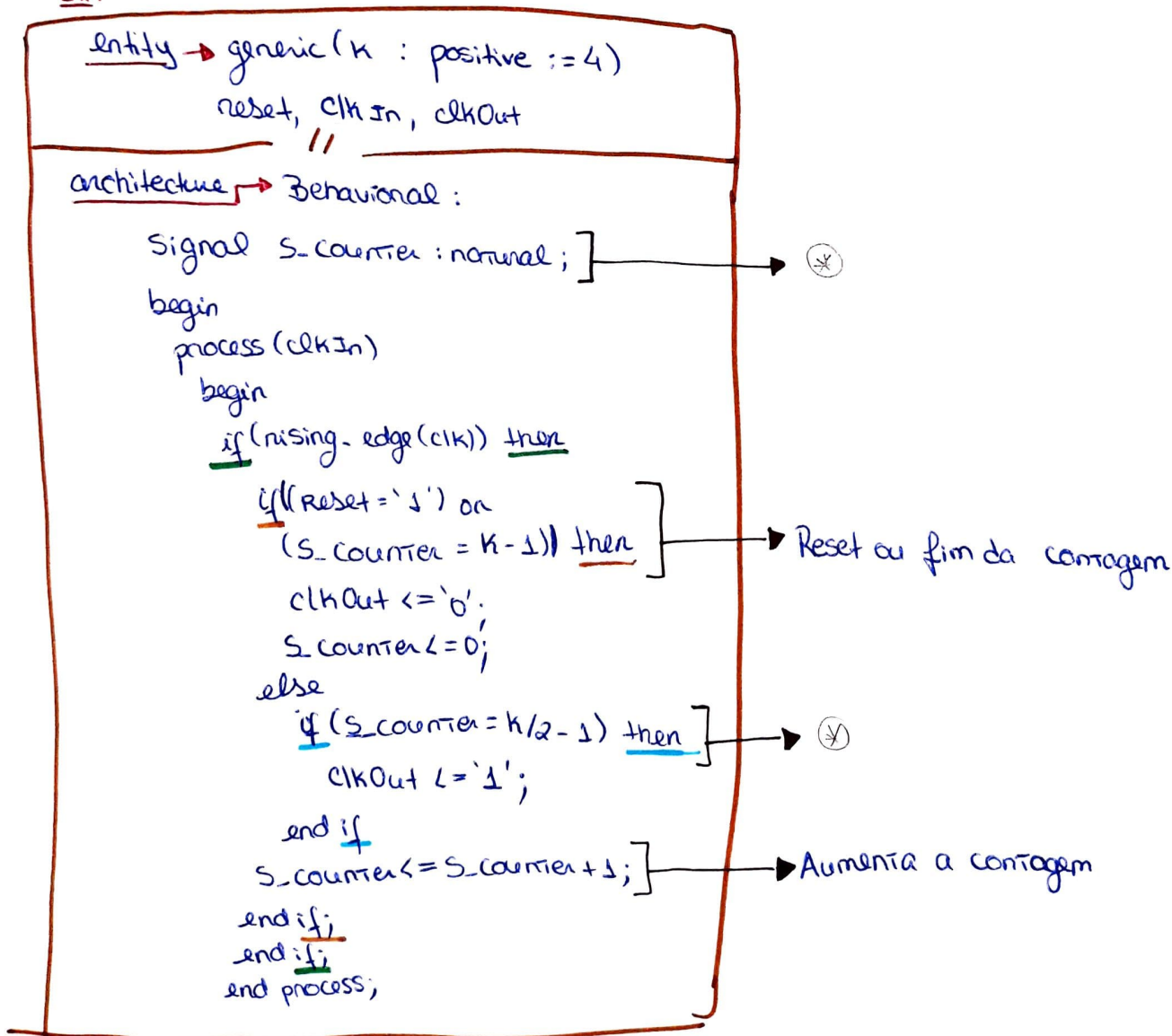
↳ Contadores

- Usamos a lógica dos operadores aritméticos (+ (up), - (down)).
- Usando os if ... then / when / ... (atribuições condicionais), podemos usar como se fosse um multiplexer para selecionar entre up, down, congelamento paralelo, "freeze", etc.

↳ Divisões de Frequência:

- A divisão da frequência de um sinal de clock por fatores inteiros 2^n pode ser efetuada por um contador.
- A divisão da frequência de um sinal de clock por fatores inteiros arbitrários (k) requer um hardware mais elaborado, baseado num contador de módulo k .

Ex:



↳ Temporizadores:

• São módulos utilizados para medir tempo, ou para gerar um evento após ter decorrido um dado intervalo de tempo (depois do temporizador ter sido iniciado).

Exemplo de implementação em VHDL:

```
entity: → generic (k : positive := 5);  
Reset, clk, enable, start, timerOut;
```

Architecture → Behavioral

```
Signal: S_count : integer := 0;
```

```
begin
```

```
assert (k >= 2);
```

```
process (clk)
```

```
begin
```

```
if (rising_edge (clk)) then
```

```
if (Reset = '1') then
```

```
timerOut <= '0';
```

```
S_count <= 0;
```

```
elsif (enable = '1') then
```

```
if (S_count = 0) then
```

```
if (start = '1') then
```

```
S_count <= S_count + 1;
```

```
end if;
```

```
timerOut <= '0';
```

```
else
```

```
if (S_count = (k-1)) then
```

```
timerOut <= '1';
```

```
S_count <= 0;
```

```
else
```

```
timerOut <= '0';
```

```
S_count <= S_count + 1;
```

```
end if;
```

```
end if;
```

```
end if;
```

```
end if;
```

```
end process;
```

→ k deve ser ≥ 2 para o contador
Ser (mínimo) de módulo 2
(0-1-0-1)

→ Inicialização

→ Teste do sinal enable

→ Se o contador estiver parado

→ Detecção de um novo disparo

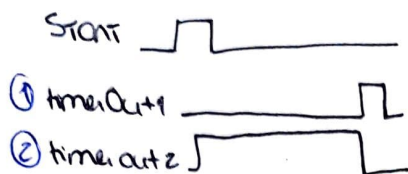
→ Desativação da saída

→ Detecção do final da contagem
e ativação da saída

→ Incremento do contador
a meio da contagem

• 2 tipos de temporizadores:

- ① - Atraso "à operação" - saída ativada após decorrido um tempo predefinido
- ② - Atraso "à desoperação" - saída desativada após decorrido um tempo predefinido



Registos e módulos de deslocamento

↳ Operações de Deslocamento

Deslocamento	operando	Resultado (1 bit)	Resultado (2 bits)
Esquerda (lógico/aritmético) introduz 0's	0 1 0 0	1 0 0 0	0 0 0 0
	0 1 0 1	1 0 1 0	0 1 0 0
À direita, lógico introduz zeros	0 0 1 1	0 0 0 1	0 0 0 0
	1 0 1 1	0 1 0 1	0 0 1 0
À direita, aritmético preserva o sinal	0 0 1 1	0 0 0 1	0 0 0 0
	1 0 1 1	1 1 0 1	1 1 1 0

Aplicações:

- Conversão de dados paralelo $\leftrightarrow$ série
- Algoritmos de detecção e correção de erros.

Nota: Deslocar i bits à esquerda $\Leftrightarrow \times 2^i$
Deslocar i bits à direita $\Leftrightarrow : 2^i$

2 abordagens / implementações

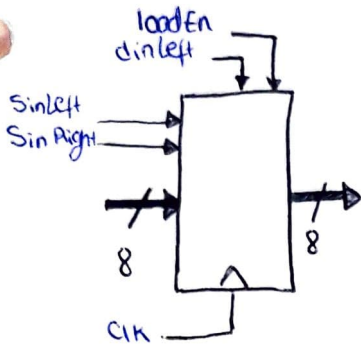
Iterativa (sequencial)

Shift Register

Paralela (Combinatória)

Barrel Shifter

↳ Registo de Deslocamento (Shift Register):



LoadEn	dinLeft
1	-
0	1
0	0

Carregamento paralelo
Desloc. para a esquerda
Desloc. para a direita

entity Iter Shifter is

```

port ( clk      : in std_logic;
      loadEn    : in std_logic;
      SInLeft   : in std_logic;
      SInRight  : in std_logic;
      dinLeft   : in std_logic;
      dataIn    : in std_logic_vector(7 downto 0);
      dataOut   : out std_logic_vector(7 downto 0));
end Iter Shifter;
```

Identificação dos pontos de entrada e saída

architecture Behavioral of Iter Shifter is

Signal s_shiftReg: std_logic_vector(7 downto 0);

```

begin
  process (clk)
  begin
    if (Rising-edge(clk) then
```

```

      if (loadEn = '1') then
        s_shiftReg <= dataIn;
```

Carregamento paralelo

```

      elsif (dinLeft = '1') then
        s_shiftReg <= s_shiftReg(6 downto 0) & SInLeft;
```

concatenação pela esquerda

```

      else
```

```

        s_shiftReg <= SInRight & s_shiftReg(7 downto 1);
```

concatenação pela direita

```

      end if;
```

```

    end if;
```

```

  end process;
```

```

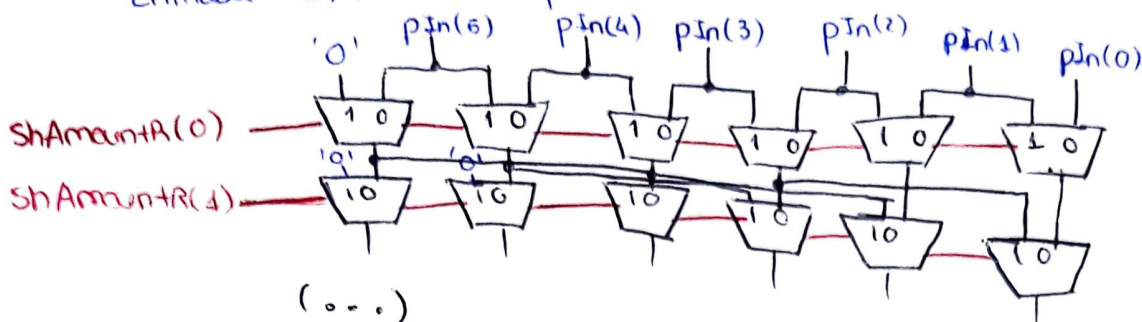
  dataOut <= s_shiftReg;
```

```

End Behavioral;
```


↳ Barrel Shifter

- Componente Combinatório
- Realiza deslocamentos à direita ou à esquerda
- Implementação com multiplexers.
- Entrada $shAmountR(i)$ provoca um deslocamento de 2^i



Em VHDL:

→ usa a função `shift-left` e `shift-right`.

→ Para deslocamentos lógicos:

`Shift-left(unsigned, integer)`
`Shift-right(unsigned, integer)`

→ Para deslocamentos aritméticos:

`Shift-right(signed, integer)`

Exemplo:

`S_shAmount <= to_integer(unsigned(shAmount));` } Quantidade de deslocamento

`process (dataIn, dirLeft, s_shAmount)`
`(...)`

`if (dirLeft = '1') then`

`dataOut <= std_logic_vector(shift-left(unsigned(dataIn), s_shAmount));`

`else`

`dataOut <= std_logic_vector(shift-right(unsigned(dataIn), s_shAmount));`

`end if;`

`(...)`

Estilos de codificação adequados à síntese

↳ Estrutura Típica de Processos Combinatórios

`process (<lista de sensibilidade com todas as entradas do processo>)`

`begin`

atribuições a sinais / pontos

Todas as combinações dos vetores de entrada devem ser especificadas para as saídas, mesmo que sejam don't care ("--"), Para evitar a inferência de latches.

`end process;`

↳ Estrutura Típica de Processos Síncronos

process (< lista de sensibilidade com sinais de clk e Set/Reset assíncronos >)

begin

< teste de sinais assíncronos >

< atribuições assíncronas >

< teste do flanco ativo do clock >

< teste de sinais síncronos >

< atribuições síncronas >

end process

normalmente, a utilização de condicionais "if... then" especifica as saídas

↳ Saídas não completamente especificadas

- Quando o valor de um sinal / ponto não é especificado para um ou mais conjuntos de entradas a ferramenta de síntese infere que esse sinal / ponto deve corresponder à saída de um elemento de memória (flip-flop e latches).

- Latches são pouco usadas, mas são frequentemente inferidas, devido a descrições incompletas não pretendidas.

Exemplo: Redução de prioridade $4 \rightarrow 2$

O sinal encodedOut é especificado para todas as combinações de entrada, exceto "0000", logo um latch é inferido a partir deste processo

```
(...)  
else  
  validOut <= '0';  
end if;
```



```
(...)  
else  
  validOut <= '0';  
  encodedOut <= "--";  
end if;
```

↳ Múltiplas atribuições a Sinais/Pontos

- É boa prática realizar apenas uma atribuição a um sinal / ponto ao longo do caminho de execução de um processo.

- No entanto, se forem realizadas várias atribuições, segundo a semântica do VHDL, prevalece a última.

- Podemos utilizar esta facilidade para tornar o código mais compacto e se não afetar a legibilidade do código.

Simulação: Construção e utilização de test benches em VHDL

↳ Simulação com HDL

- É fundamental para validar o modelo de um sistema desde as fases iniciais de um projeto até à implementação.
 - Económica
 - Muito controlável
- Útil para observar qualquer ponto do sistema, por vezes inacessível na implementação em hardware (devido a altas frequências, por exemplo).

↳ Simulação em VHDL

- Baseada em **testbenches** (bancos de ensaio).
- São módulos onde o modelo VHDL a simular (Unit Under Test - UUT) é instanciado e onde são aplicados estímulos (vetores de simulação) para validar o comportamento.
- Uma testbench atua como top level no Simulador.
- Pode ser construída de duas formas

Gráfica:

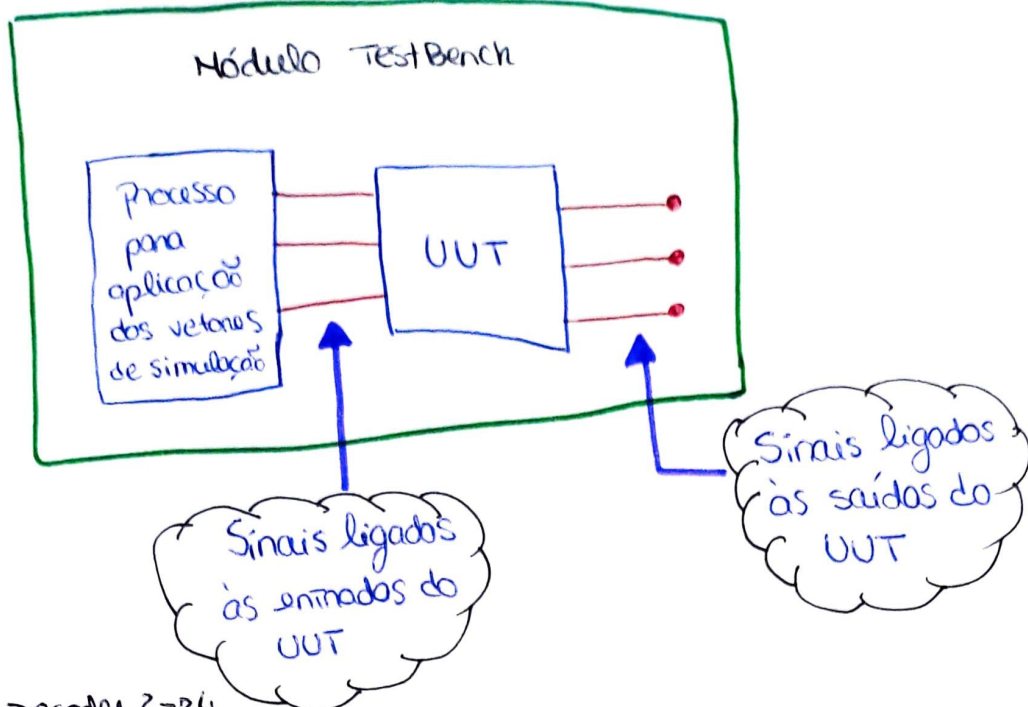
- Através do ficheiro VWF e aplicação com GUI (Graphical User Interface)
- "Amanhada" a uma ferramenta específica

Textual:

- Como um ficheiro VHDL, mas com uma estrutura específica.
- Ficheiros portáteis/independentes da ferramenta

↳ Estrutura típica de uma Testbench para um componente Combinatório

- Entidade sem portas
- Arquitetura:
 - Instanciación do UUT no corpo da arquitetura
 - Declaração dos sinais a ligar aos pontos do UUT na parte declarativa da arquitetura.
 - Definição de um processo para aplicar os vetores de simulação ao longo do tempo.
 - ⊗ Em sistemas mais complexos pode ser usado mais que um processo para este efeito.



Exemplo: Decoder 2-24

<libraries>

```
entity Dec2_4Entb is
```

entidade sem portas

```
end Dec2_4Entb;
```

architecture Stimulus of Dec2_4Entb is

Signal s-enable : std_logic;

Signal s-inputs : std_logic_vector(1 downto 0);

Signal s-outputs : std_logic_vector(3 downto 0);

Sinais para ligar
aos pontos da UUT

uut : entity work.Dec2_4Ent (Behavioral)

port map (...);

ligação dos pontos da UUT aos sinais
criados anteriormente

stim_proc : process

begin

wait for 100 ns;

s-enable <= '0';

wait for 100 ns;

s-enable <= '1';

wait for 100 ns;

s-inputs <= "00";

(...)

end process;

end Stimulus;

Vetores de teste

A construção "wait for" é suportada
apenas para a simulação.

Esta compilação é
Realizada no software
"ModelSim ALTERA"

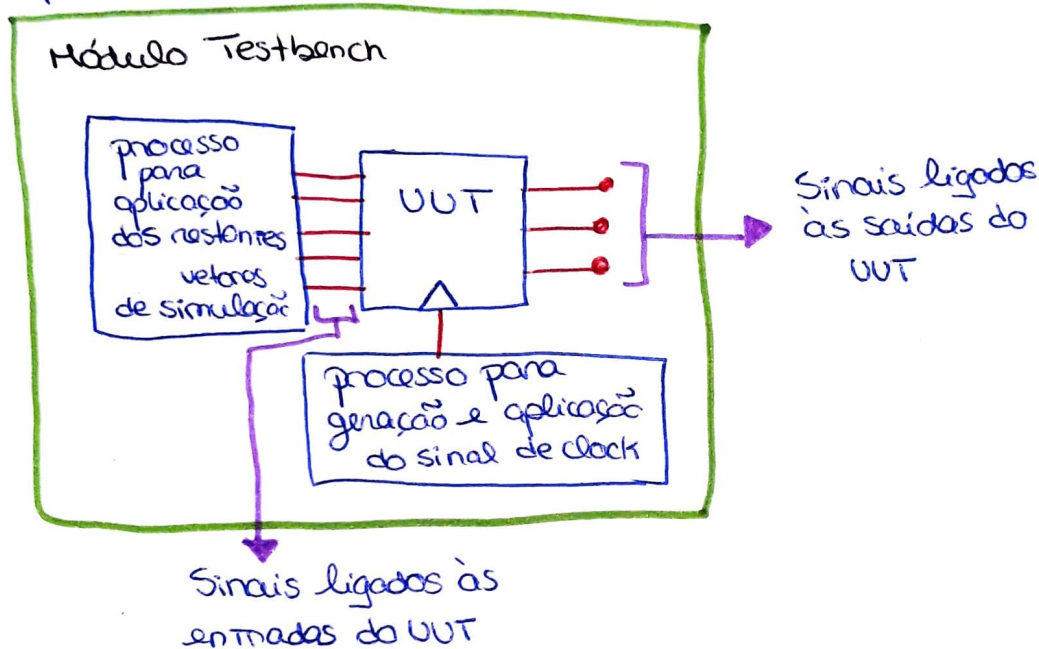
↳ Estrutura de uma Testbench para um Componente Sequencial

- Entidade sem portas.

- Arquitetura :

- Instanciação da UUT no corpo da arquitetura.
- Declaração dos sinais a ligar aos portos da UUT na parte declarativa da arquitetura.
- Definição de um processo para geração do sinal de clock.
- Definição de um processo para aplicação dos vetores de simulação ao longo do tempo.

⊗ Em sistemas mais complexos pode ser usado mais do que um processo para este efeito.



Exemplo: Contador Up/Down de 4 bits

<libraries>

entity BinUDCntEnRst4Tb is
end BinUDCntEnRst4Tb;] → Entidade sem portas

architecture Stimulus of BinUDCntEnRst4Tb is

signal S_reset, S_clk : std_logic;
signal S_enable, S_updown_n : std_logic;
(...)

] → Sinais internos da Testbench

begin

uut: entity work.BinUDCntEnRst4 (Behavioral)
portmap (...);] → Instanciação da UUT

clock_proc: process

begin
S_clk <= '0'; wait for 100ns;
S_clk <= '1'; wait for 100ns;
end process

↓
Sinal de clock com período 200ns

Stim_proc: process

begin
S_reset <= '1';
S_enable <= '0'; S_updown_n <= '1';
wait for 300ns;

(...)

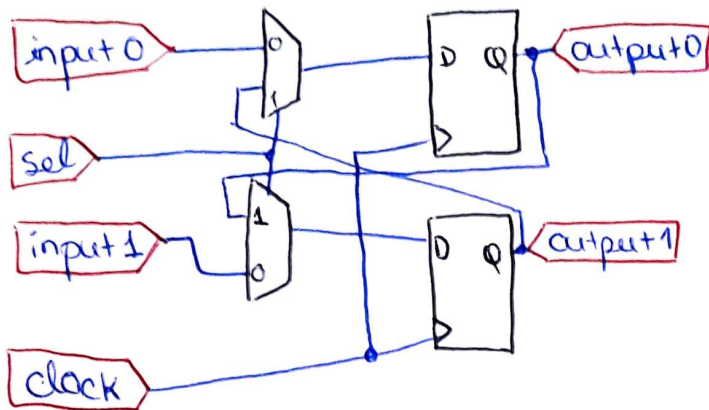
end process;

end Stimulus;

vetores de teste

Princípios Básicos de Simulação

↳ Modelação do Paralelismo do Hardware



No flanco ascendente do clk:

- Quando $sel = '0'$

$output 0 \leftarrow input 0$

$output 1 \leftarrow input 1$

Quando $sel = '1'$

$output 0 \leftarrow output 1$

$output 1 \leftarrow output 0$

1ª abordagem:

- usar apenas 1 processo, colocando o clock na lista de sensibilidade

2ª abordagem:

- Dividir em dois processos, pois a ordem destes em VHDL é irrelevante. O clock tem de constar na lista de sensibilidade dos dois processos.

↳ Hardware vs Simulação do Modelo VHDL

• Hardware real

→ Todos os módulos operam em paralelo

→ Os atrasos são impostos pela tecnologia e projeto do sistema

• Ambiente de simulação

→ Baseado em ferramentas de simulação de eventos discretos a executar sobre processadores de uso geral.

→ Pode ser realizados a vários níveis:

- Ideal - Sem atrasos (Comportamental)
- PÓS-síntese - sem atrasos (Funcional)
- com atrasos, pós-implementação (Temporal)

→ Resolução muito mais fina (ciclos de simulação muito inferiores)

→ Paralelismo:

- Atribuições concorrentes
- Processos

↳ Processos e Listas de Sensibilidade

- As listas de sensibilidade são apenas uma forma de otimizar o desempenho da simulação, isto é, para evitar execuções desnecessárias de processos no simulador.
- O comportamento de um processo deve ser o mesmo com ou sem lista de sensibilidade.
- As ferramentas de síntese são capazes de detetar sinais em falta nas listas de sensibilidade.
- As listas de sensibilidade não têm qualquer influência na síntese do sistema.
- Uma vez que o código é sintetizado, a execução na FPGA pode ter resultados distintos do comportamento da simulação, simplesmente porque a lista de sensibilidade não estava completa.

Síntese de Máquinas de estado finitas

↳ Passagem da especificação à implementação

- Conceção de uma solução para um problema concreto (muitas vezes) inicialmente descrito em linguagem natural.
- Formalização / abstracção segundo o modelo de FSM / MEF
- Processo composto por diferentes passos de modelação, optimização e geração de hardware
- Solução frequentemente não única e sub-ótima

↳ Instrumentos / Ferramentas de modelação e síntese de FSM

- Diagramas de estados

- Contas ASM (Algorithmic State Machines)
- Tabelas de estado/saídas
- Tabelas de transição/saídas
- Tabelas de excitação

- Diagramas Temporais

- Linguagens de descrição de Hardware

↳ Modelação com o editor do Quantus Prime

É necessário ter atenção a certos aspetos:

- As condições de transição têm de ser mutuamente exclusivas.
- As condições não especificadas correspondem à manutenção do estado
- É possível gerar um ficheiro VHDL a partir do ficheiro smf (máquina de estados)

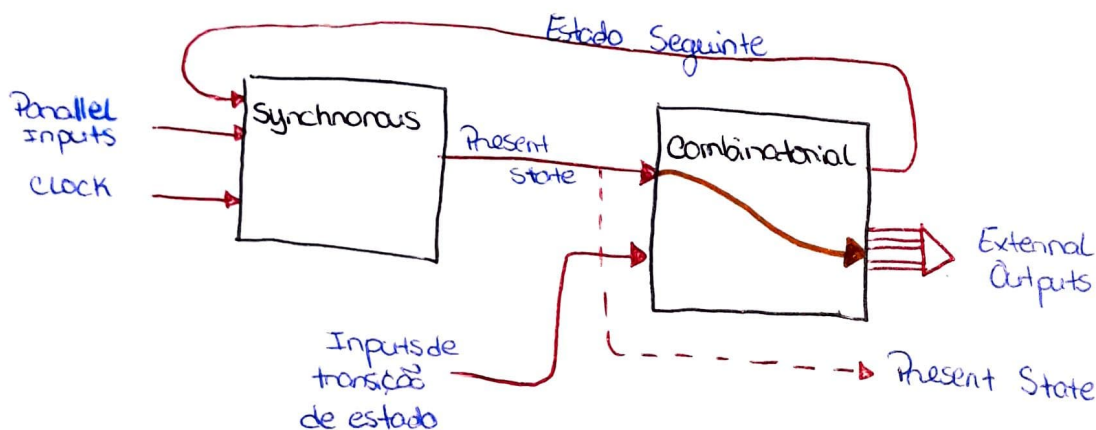
↳ Estilo de codificação baseado em "2" processos - Máquinas de Moore

Processo "Synchronous":

- Atribuições dependentes de um evento de relógio e/ou de atribuição / inicialização assíncrona dos elementos de memória

Processo "combinatorial":

- Atribuições relacionadas com a determinação de:
 - saídas
 - estado seguinte



↳ Síntese da FSM

- Realizada pelo compilador / ferramenta de síntese
 - Codificação dos estados com base nos estados simbólicos
 - Geração do registo de estado (com o número de bits necessários em função da codificação dos estados).
 - Determinação e otimização dos circuitos combinatórios de estado seguinte e das saídas.

NOTA: No Quantus Prime é também possível forçar a técnica de codificação de estados em

Assignments → Settings → Compiler Settings → Advanced Settings (Synthesis)
→ State Machine Processing

Ex: Gray, Johnson, One-hot, ...

↳ Máquinas de Mealy

- O estilo "2 processos" interdependentes pode facilmente adaptar-se às máquinas de Mealy
 - Dependendo da forma como as saídas são atribuídas (no processo combinatório)
 - Moore: dependem apenas do estado (atribuídas diretamente nos when de um case).
 - Mealy: dependem do estado e das entradas (atribuídas em if...then...else dentro dos when de um case)
- Cuidado a ter com as saídas:
 - Garantir sempre a atribuição (processo combinatório)

↳ Sistema Computacional

- Datapath (unidade de execução)
 - Elementos operativos, de encaminhamento e de armazenamento
- Controlpath (unidade de controlo)
 - Máquina de estados finita

Interligação entre controlpath e datapath:

- Sinais de controlo (controlpath → datapath)
- Sinais de estado (controlpath ← datapath)

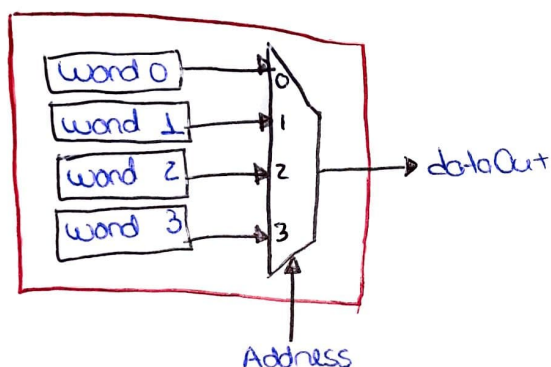
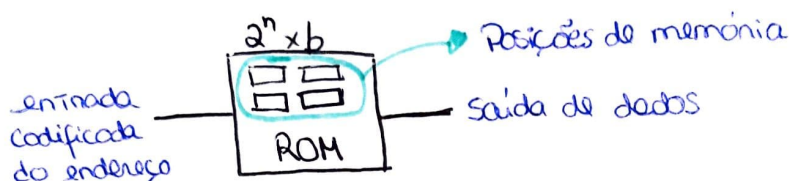
↳ Máquinas de Estado Finitos Comunicantes

- Partição da funcionalidade do sistema em 2 ou mais FSM paralelas ou sequenciais.
 - Decomposição visa facilitar o desenvolvimento e a validação
 - Partilha de sinais de inicialização e sincronização
 - Frequentemente operam em diferentes flancos do mesmo sinal de clock

Memórias de um ponto e multi-ponto

↳ Read Only Memory (ROM)

- Memória apenas de leitura
- Num dado instante pode ser lida qualquer posição de memória
- Útil para armazenamento de informação de forma fixa / não volátil
 - Tabelas / Lookup tables estáticas
 - Instruções codificadas
 - Mensagens, Strings, etc.
- O conteúdo é definido quando do fabrico ou programação do dispositivo.



Em VHDL...

entity ROM_8x4 is

port (address : in std_logic_vector(2 downto 0);
dataOut : out std_logic_vector(3 downto 0));

end ROM_8x4;

architecture Behavioral of ROM_8x4 is

subtype TData word is std_logic_vector(3 downto 0);

type TROM is array (0 to 7) of TData word;

constant c_memory : TROM := ("0000", "0001", ...);

begin

dataOut <= c_memory(to_integer(unsigned(address)));

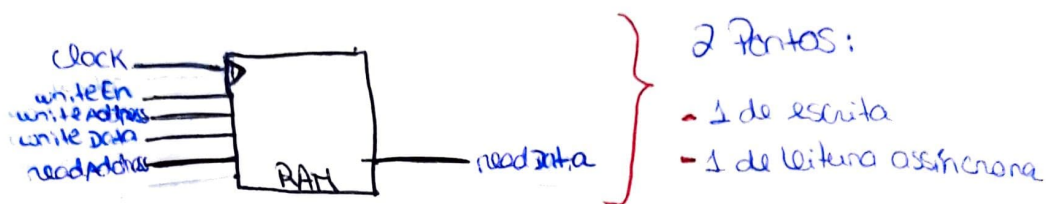
end Behavioral

↳ Subtype define um conjunto restringido de valores do seu tipo base

↳ tipo array: definir o intervalo e o tipo de dados

↳ Random Access Memory (RAM)

- Memória de leitura/escrita
- Num dado instante pode ser lida ou escrita qualquer posição de memória
- Útil para armazenamento de informação arbitrária que pode ser alterada durante o funcionamento do sistema.
- Pode ser de vários tipos (estática, dinâmica, ...)
- Pode disponibilizar:
 - 1º Ponto: apenas uma operação de leitura ou escrita num dado momento (caso mais frequente).
 - Multi-ponto: várias operações de leitura e/ou escrita num dado instante.



Em VHDL...

```
begin
  process (clk)
  begin
    if (rising_edge(clk)) then
      if (writeEnable = '1') then
        S-memory(to_integer(unsigned(writeAddress))) <= writeData
      end if;
    end if;
  end process;

  ReadData <= S-memory(to_integer(unsigned(readAddress)));
```

Escrita
Síncrona

Leitura
Assíncrona

Nota: Uma RAM pode ainda ter 3 pontos (ou mais)

- 2 de leitura
 - 1 de escrita
- } Por exemplo

- A leitura também pode ser síncrona (2 processos)

↳ Parametrização de uma memória

- Os parâmetros de uma memória podem ser, por exemplo:
 - Profundidade: n° de palavras (n° de bits do endereço codificado)
 - Longura: n° de bits das palavras

Nota: $2^n \rightarrow (2^{**}n)$ em VHDL

Arquitetura de Computadores

- Nível de abstração diferente
- Blocos funcionais complexos:
 - Unidades de entrada: receção de informação vinda do exterior
 - Unidades de saída: envio de resultados para o exterior
 - Memória: armazenamento de
 - programas
 - dados para processamento
 - resultados
 - CPU: processamento da informação através da execução do programa armazenado em memória
- Cada um destes blocos é um sistema digital

↳ Níveis de Representação

- Computadores executam comandos (instruções);
- Conjunto de instruções: operações que um computador suporta;
- Instruções: coleção de bits
Exemplo: 0010100111000000 → Somar dois valores
- Linguagem de máquina: instruções de mais baixo nível (0's e 1's)
- Assembly: Linguagem de montagem. Notação legível por humanos para o código de máquina que uma arquitetura de computador específica usa
Exemplo: ADD \$2, \$3, \$4
- Assembler: "Montador". Traduz o código assembly para o código de máquina.
- Assembly obriga o programador a pensar como um computador (uma instrução por cada ação a executar). Não é produtivo e requer conhecimento detalhado da máquina.
- Linguagens de alto nível: C, C++, Java
- Compilador: traduz o código fonte de uma linguagem de alto nível para uma de baixo nível.

↳ Instruções

• Independentemente do tipo de CPU e da sua estrutura interna, qualquer instrução deverá ter uma codificação que permita responder às seguintes questões:

- Qual a operação a realizar?
- Qual a localização dos operandos?
- Onde colocar o resultado?
- Qual a próxima instrução?

• As instruções são guardadas na memória e endereçadas com um contador, chamado Program Counter

